LLM/GenAI and Education:
Knowledge Complexity Model and Governed Development of Competences

System and Software Engineering: Education and Practice

From DevOps to DevGenOps and Agentic Software Development

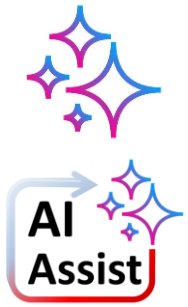
Brief introduction to initiate thinking

Yuri Demchenko

IvI UvA, NTUU “Igor Sikorsky KPI”

Outline and Concepts – Planned structure

- Software Complexity and Engineering Governance (SCEG) model
- How University and VET curricula should respond
  - Using SCEG for System and Software Engineering curriculum design
  - New model for Agentic AI competences, skills and Body of Knowledge (BoK) Groups
- Redefining Bloom's Taxonomy to AI-Assisted Engineering Competence Taxonomy (AECT)
- Body of Knowledge Groups/Clusters and prompt construction aspect
- Additional Information
  - Knowledge-Complexity and Epistemic-Governance Model of LLM-Assisted Research (KCEG)



Sign indicating Generated with GenAI/ChatGPT

or Assisted with ChatGPT (AGI)

Disclaimer

This research and development is based on and bult around the concept of Knowledge.

- Knowledge doesn't have the measurable properties similar to (research) experiments.
- Knowledge has variety of presentation and expression forms
- But knowledge can be validated with the epistemic correctness – attributed only to humans/researchers and dependent on their personal epistemic model

Consequently, presented here information (working knowledge) may have variable form, however clearly representing the author's epistemic model of the research topic

Setting up a playground

- Conceptual chain for Human - AI interaction for HGLEP/KCEG
- Human – AI interface
- Knowledge Complexity: Presentation – Representation - Perception

Conceptual chain for Human - AI interaction for HGLEP/KCEG

1. Knowledge source
- paper, dataset, prior dialogue, experiment, model, human expertise

2. Knowledge presentation
- text, diagram, table, formula, code, LLM response

3. Knowledge representation (inference/actionable knowledge use)
- concepts, relations, ontology, claims, evidence, assumptions, methods

4. Knowledge perception & reconstruction
- human or AI agent interprets and reconstructs the meaning

5. Knowledge assessment
- KCEG evaluates level, completeness, uncertainty, reliability, novelty

6. Internalisation & agent memory update
- human updates personal epistemic model
- AI agent updates context, agent system memory, graph, or retrieval state

7. Knowledge action
- explanation, critique, decision (to act), experiment, paper writing, new prompt

8. Regeneration
- human or AI generates a new presentation

Knowledge is an actional element of the human cognitive composition
- Knowledge structure/direct actions
- Built-in/constructed/re-constructed knowledge structure defines learning efficiency and effective curricula

Basis for a generative loop:
presentation → perception → representation → assessment → memory → action → new presentation

Knowledge Presentation – Representation - Perception

Available/Received Knowledge

Knowledge Presentation

1. Vocabulary/Terminology complexity (KC1)
2. Semantic complexity (KC2)
3. Ontological complexity (KC3)
4. Structural complexity (KC4)
5. Methodological complexity (KC5)
6. Epistemic complexity/validity (KC6)

Inferential/LLM knowledge representation (internal)

Knowledge Representation

1. Vocabulary / terminology (K1)
2. Syntax / expression form (K2)
3. Factual / propositional layer (K3)
4. Semantic layer (K4)
5. Ontological layer (K5)
6. Structural / graph layer (K6)
7. Inferential layer (K7)
8. Methodological layer (K8)
9. Epistemic / research-frontier layer (K9)

Perception via Information, Epistemic Validation, Internalisation

Knowledge perception

- If a human doesn't have a full Kx or KCx model of KCEG, they may treat received/observed information structured wrongly or a kind of imaginary mixed
- This is the case for children
  - Can we control and speed up development?
  - Can we do the same for Doctoral (PhD) Students?
- Children imaginary treating objects and animals with human features
  - Hypothetically because of KCEG
  - Agent-centered perception model by children
- TODO: Adding Pragmatics to Semantics Complexity level as related to language understanding by humans

KCEG Knowledge Complexity Perception by Expert and Novice

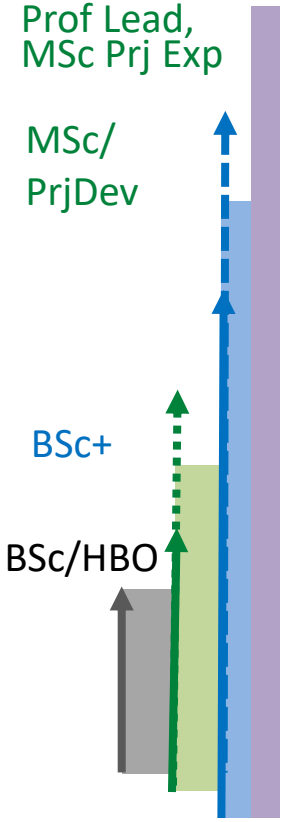
Mechanism: Perception is holistic, understanding is selective reconstruction

- **1. Real object/event/system**
  The full THING EXISTS with many properties: visible, functional, structural, causal, methodological, historical, epistemic.
- **2. Sensory/perceptual intake**
  The person receives SIGNALS: visual form, words, diagrams, behaviour, motion, numbers, names, artefacts.
  - ----------
- **3. Cognitive reconstruction**
  The MIND MAPS signals into existing *schemas*: vocabulary, concepts, relations, structures, methods, causal models, epistemic status.
- **4. Expressed understanding**
  The person explains, draws, predicts, classifies, evaluates, or acts.
- **Hypothesis: Gap between layers 2 and 3/4**.
  - The person may "see" the whole event, but their **model of what is significant** is constrained by their available cognitive schemas.
  - This is close to the expert-novice literature review: experts detect meaningful patterns that novices overlook, and experts organise knowledge around deep principles rather than surface attributes.

Applying KCEG Methodology to Software Complexity Model (SCEG)

=> To provide a basis for System and Software Engineering Curriculum

code → modules → data models → APIs → architecture → quality attributes → runtime operation → lifecycle governance



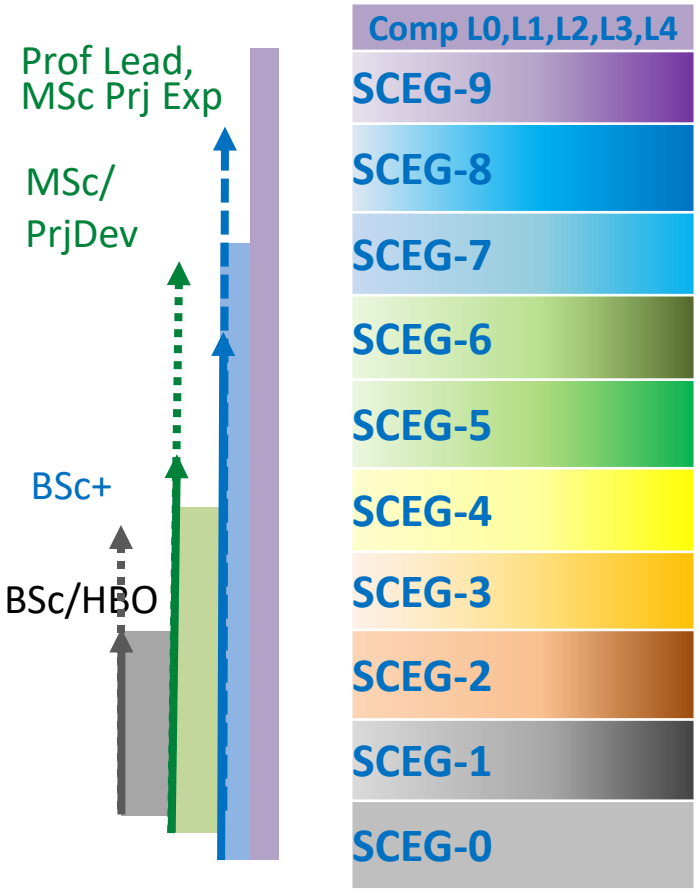
- SCEG-9 Evolution, governance, compliance, sustainability
- SCEG-8 Runtime, deployment, DevOps -> MLOps, CI/CD, observability
- SCEG-7 Quality attributes, verification, security, risk
- SCEG-6 Architecture styles, layers, views, decisions
- SCEG-5 APIs, contracts, data models, semantic interoperability
- SCEG-4 Modules, components, OO/functional design, patterns
- SCEG-3 Functional semantics, requirements traceability
- SCEG-2 Algorithms, data structures, computational logic
- SCEG-1 Code syntax, local implementation, style
- SCEG-0 Prerequisite: Intent, requirements, constraints, acceptance criteria

Course design focus

- Sequential courses: SCEG-1 – SCEG-3
- Progressive recurrent topics/courses SCEG-4 – SCEG-8

Software Complexity Model (SCEG) as a basis for System and Software Engineering Curriculum with an indication of Competence Levels

code → modules → data models → APIs → architecture → quality attributes → runtime operation → lifecycle governance



Comp L0,L1,L2,L3,L4
SCEG-9
SCEG-8
SCEG-7
SCEG-6
SCEG-5
SCEG-4
SCEG-3
SCEG-2
SCEG-1
SCEG-0

- SCEG-9 Evolution, governance, compliance, sustainability
- SCEG-8 Runtime, deployment, DevOps -> MLOps, CI/CD, observability
- SCEG-7 Quality attributes, verification, security, risk
- SCEG-6 Architecture styles, layers, views, decisions
- SCEG-5 APIs, contracts, data models, semantic interoperability
- SCEG-4 Modules, components, OO/functional design, patterns
- SCEG-3 Functional semantics, requirements traceability
- SCEG-2 Algorithms, data structures, computational logic
- SCEG-1 Code syntax, local implementation, style
- SCEG-0 Prerequisite: Intent, requirements, constraints, acceptance criteria

Course design focus
- Sequential courses: SCEG-1 – SCEG-3
- Progressive recurrent topics/courses SCEG-4 – SCEG-8

→ Knowledge/Competence at Level C3+

⇢ Knowledge/Competence at Level C1+ Awareness

SCEG Competence and Role Profiling Model — SCEG-CRP

SCEG levels describe increasing forms of software-system complexity, but practitioner profiles need not progress through them sequentially. Professional competence is represented by the proficiency demonstrated independently at each SCEG level and by the ability to integrate related levels when performing a role.

- Foundational components
  - SCEG complexity model: S0–S9
  - Competence scale: C1–C5
  - Agentic work mode and governed agentic coding: A0 – A4
- Practical implementation
  - Evidence record for each assessed cell
  - Individual competence profile
  - Target role profile
  - Team coverage profile
  - Course or development transformation profile

Competence Levels for each of SCEG Level

Competence	Proposed name	Characteristic capability
C1	Aware	Recognises concepts, terminology, artefacts and common problems
C2	Contributing	Performs bounded tasks using established guidance, examples and review
C3	Independent	Completes, explains, verifies and corrects normal professional work independently
C4	Owning and integrating	Owns a subsystem or practice, handles difficult cases, integrates work across boundaries and guides others
C5	Evolving and governing	Defines approaches, evaluates systemic trade-offs, establishes organisational practice and evolves the capability

- Competences expanding to practical and expert professional work
- Professional profiles/roles may not require a full competence level for each SCEG level
- Target MSc competence level C3
  - To be a subject of assessment

Example Competence profiles for typical roles

Role	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9
Application developer	C (C2)	O	I	I	I	C/I	C	I	C	A (C1)
Backend/service engineer	C (C2)	O	I	O	O	O	I	I	I	C
DevOps/platform engineer	C (C2)	I	C	C	C	I	I	O (C4)	O/G	I
SRE	C (C2)	I	I	I	C	I	I	O (C4)/ G (C5)	O/G	I/O
Test/quality engineer	C (C2)/ I (C3)	C	C	O	I	I	I	O (C4)/ G (C5)	I	C/I
Software architect	O (C4)	I	C/I	O	O	O	G	O	I/O	O
Technical lead	I/O	O	I	O	O	O	O	O	I/O	I/O
Engineering manager	O	C	A (C1)/ C (C2)	I	I	I	O	O	I	O/G

- Competences expanding to practical and expert professional work

- **Legend:**
  A = C1 awareness, C = C2 contributing, I = C3 independent, O = C4 owning/integrating, G = C5 governing/evolving, – = normally not central.

Competence Assessment Dimensions

- Understanding
  - Does the person understand the concepts, assumptions and relationships?
- Performance
  - Can the person produce or modify the required artefacts?
- Verification
  - Can the person determine whether the result is correct and adequate?
- Autonomy and responsibility
  - How independently and accountably can the person work?
- Transfer and improvement
  - Can the person adapt the knowledge, guide others and improve practice?

For AI-assisted development, **tool operation and prompting are not sufficient evidence of competence**. Competence requires understanding, verification, correction and responsibility for the resulting software.

Evidence-based assessment & Specific for AI-assisted work

Evidence category	Examples
Produced artefacts	Code, API specifications, schemas, architecture views, tests, pipelines, dashboards
Engineering decisions	ADRs, trade-off analyses, risk assessments, migration decisions
Verification evidence	Reviews, test results, incident diagnosis, conformance checks
Operational evidence	Deployments, rollback execution, monitoring improvements, incident reports
Explanation	Design walkthrough, oral defence, review discussion, technical documentation
Improvement and influence	Coaching, reusable patterns, standards, process or platform improvements

- Each competence rating should require evidence appropriate to the SCEG domain.
- For AI-assisted work, additional evidence should show that the practitioner:
  - identified assumptions in the generated result;
  - checked it against requirements and architecture;
  - tested or analysed its behaviour;
  - corrected significant deficiencies;
  - can explain why the final result is acceptable;
  - assumes professional responsibility for it.
- A chat transcript may support the assessment, but it does not by itself demonstrate competence.

SCEG and Competences Model for Agentic SSE

Layer 1 — SCEG complexity model

SCEG continues to describe the objects and relationships whose complexity must be mastered:

- requirements and intent;
- local code and algorithms;
- functional semantics;
- modules and components;
- APIs and data models;
- architecture;
- quality and risk;
- deployment and operations;
- evolution and governance.
- This layer should change slowly and only when the conceptual structure of software engineering changes—not when a new coding product is released.

Layer 2 — SCEG competence profile

The C1–C5 competence scale continues to describe human capability:

- C1 Aware;
- C2 Contributing;
- C3 Independent;
- C4 Owning and integrating;
- C5 Evolving and governing.

Competence descriptions should become **tool-neutral**.

Layer 3 — Agentic work mode

Each competence cell can optionally carry an agentic-work qualifier:

- A0 Direct
  - The practitioner performs the activity directly, without substantive AI assistance
- A1 AI-assisted
  - AI supports explanation, completion, analysis or limited generation
- A2 Agent-delegated
  - A bounded engineering task is delegated to an agent and reviewed by the practitioner
- A3 Agent-orchestrated
  - The practitioner coordinates long-running or multiple agents and integrates their outputs
- A4 Agent-governed environment
  - The practitioner designs organisational guardrails, platforms, policies and assurance systems for agentic work

Example Profiles

- Developer profile S5-C3-A2
  - Independently competent in APIs and data models, normally working by delegating bounded API tasks to an agent and verifying the result.
- Architect profile S6-C4-A3
  - Owns and integrates architectural work while coordinating several agent activities.
- Platform leader profile S8-C5-A4
  - Defines and governs the organisational environment through which agents produce, test and deploy software.

BSc and MSc to reflect Agentic coding/design competences

BSc level

The BSc profile should combine **direct formation of engineering judgement** with bounded AI-assisted work.

Bachelor students should develop a substantial direct competence floor, particularly around:

- S1 local implementation;
- S2 algorithms and data structures;
- S3 functional semantics;
- S4 modules and components;
- basic S5 interfaces and data models;
- basic testing and deployment.

Agent use can be allowed, but students should **demonstrate** they can:

- read and explain generated code;
- modify it without blind regeneration;
- identify defects;
- write or justify tests;
- trace behaviour to requirements;
- complete selected bounded tasks without an agent.

MSc level

The MSc target moves increasingly toward **system-level control of agent-assisted production**, but it should remain grounded in sufficient implementation competence.

Master students should place greater emphasis on:

- problem decomposition;
- requirements ambiguity;
- API and semantic consistency;
- architecture and architectural conformance;
- quality-attribute trade-offs;
- verification strategies;
- agent orchestration;
- CI/CD and observability;
- socio-technical risk and governance.

Masters should be able **to coordinate agents across several SCEG levels** and **identify inconsistencies** such as:

- correct code implementing the wrong requirement;
- individually valid components violating the architecture;
- passing tests that fail to cover important risks;
- operationally deployable software with unacceptable security or reliability properties.

System and Software Engineering Body of Knowledge (BoK) as a basis for Efficient Human Governed Agentic Coding

Industry recognised

- CSBOK by ACM
- SWEBOK by IEEE
- PMBOK by PMI
- DMBOK by DAMA
- NICE Framework for Cybersecurity by NIST
- SFIA (Skills Framework for the Information Age)
- e-CF (e-Competence Framework) standardised as EN 16234-1 by CEN

Domain specific, for specific technology domains

- GES – CF (Green and Environmental Sustainability Competence Framework)
- EDSF – EDISON Data Science Framework

Redefining Bloom's Taxonomy for AI assisted SSE

- **Bloom's Taxonomy for cognitive development**
  - Remember
  - Understand
  - Apply
  - Analyse
  - Evaluate
  - Create

- **AI-Assisted Engineering Competence Taxonomy — AECT**
  - A1 Recognise and Attribute
    Identify what exists and where it came from
  - A2 Explain and Trace
    Explain how and why the result works
  - A3 Apply and Direct
    Use AI purposefully to perform a defined engineering task
  - A4 Analyse and Diagnose
    Inspect AI-produced artefacts critically
  - A5 Verify and Validate
    Establish justified confidence in the result
  - A6 Improve and Integrate
    Transform AI output into an engineered system
  - A7 Govern, Defend and Transfer (new)
    Take professional ownership of the complete human–AI process

The essential distinction: artefact quality versus demonstrated competence

- AECT requires at least two separate assessment dimensions.

Dimension A — Quality of the engineering artefact
Dimension B — Demonstrated human competence

AI-Assisted Engineering Competence Taxonomy (AECT) Redefining Bloom's Taxonomy for AI assisted SSE

AECT level	Core competence	Student must demonstrate	Typical evidence	Correspondence with Bloom Taxon
A1 Recognise and Attribute	Identify what exists and where it came from	Recognise concepts, technologies, generated components, external sources and AI contributions	AI-use declaration; component inventory; terminology check; provenance record	**Remember**
A2 Explain and Trace	Explain how and why the result works	Explain code, algorithms, data flow, interfaces, dependencies and design choices; trace behaviour from requirement to implementation	Code walkthrough; annotated diagrams; oral explanation; trace questions	**Understand**
A3 Apply and Direct	Use AI purposefully to perform a defined engineering task	Formulate a task, provide context and constraints, direct an agent, select useful output and reject irrelevant output	Task specification; selected prompts; agent instructions; acceptance/rejection log	**Apply**
A4 Analyse and Diagnose	Inspect AI-produced artefacts critically	Decompose the solution; identify assumptions, inconsistencies, architectural violations, defects and missing cases	Static analysis; architecture conformance review; defect diagnosis; comparison of alternatives	**Analyse**
A5 Verify and Validate	Establish justified confidence in the result	Design tests independently of the generated solution; verify requirements, correctness, robustness, security and quality attributes	Requirements–test matrix; test rationale; negative tests; defect evidence; validation report	**Evaluate**
A6 Improve and Integrate	Transform AI output into an engineered system	Correct, refactor, optimise and integrate generated artefacts; resolve conflicts between requirements, architecture, code, tests and deployment	Before/after comparison; commits; refactoring rationale; integration tests; performance evidence	**Create**
A7 Govern, Defend and Transfer	Take professional ownership of the complete human–AI process	Justify major decisions, manage risks and provenance, adapt the solution to a new constraint, defend it orally and accept responsibility for the result	Design defence; live modification; risk register; ADRs; transfer task; reflective engineering judgement	**Extends Bloom**

Refactoring Bloom's Taxonomy (BT) to AECT AI-Assisted Engineering Competence Taxonomy

Recognise → Explain → Direct → Analyse → Verify → Improve → Govern

Linking to Learning Outcomes (preserving the cognitive progression of Bloom's Taxonomy)

- Know what the AI produced.
- Understand how it works.
- Direct the AI purposefully.
- Critically inspect the result.
- Establish evidence that it is correct and fit for purpose.
- Transform it into a better engineered solution.
- Defend, adapt and take responsibility for it.

The assessed object is not only the software artefact but the complete evidence chain:

- **student intention → AI delegation → generated candidate → human analysis → independent verification → improvement → professional ownership**

Artefact quality versus demonstrated competence

AECT requires at least two separate assessment dimensions.

- **Dimension A — Quality of the engineering artefact**
  - This includes:
  - functional correctness;
  - architecture and design;
  - code quality;
  - testing;
  - security;
  - performance;
  - documentation;
  - deployment and integration.
- **Dimension B — Demonstrated human competence**
  - This includes:
  - understanding;
  - task formulation;
  - critical analysis;
  - independent verification;
  - improvement decisions;
  - traceability;
  - oral defence;
  - professional responsibility.

	Strong human competence	Weak human competence
Strong product	Desired result: competent AI-assisted engineering	Dangerous false positive: polished product without demonstrated mastery
Weak product	Partial competence: student understands problems but has not completed the engineering work	Unsatisfactory result

Observable verbs in Assignments' narrative and Structure/Rubrics

Assignments and rubrics should use verbs that expose human competence.

- **Weak verbs**
  - use AI;
  - generate code;
  - produce an application;
  - submit prompts;
  - create tests;
  - document the interaction.
- These describe activity or output, but not mastery.

- **Strong AECT verbs**
  - **identify** AI contributions;
  - **explain** generated components;
  - **trace** requirements to code and tests;
  - **justify** delegation decisions;
  - **compare** generated alternatives;
  - **diagnose** defects and inconsistencies;
  - **verify** behaviour independently;
  - **validate** fitness for purpose;
  - **refactor** and improve generated code;
  - **defend** architectural decisions;
  - **adapt** the solution to changed constraints;
  - **account for** residual risks and uncertainty.

Typical prompt design elements -- SCEG Levels 1-5

SCEG-1 — Local Code Expression
Focus: syntax, language constructs, basic implementation.
The AI agent can generate small functions, scripts, classes, or utilities.
Typical prompt elements:
- Use Python 3.12. Implement a function that validates email addresses. Include type hints, docstring, and three examples.

SCEG-2 — Algorithmic and Data-Structure Logic
Focus: correct computational logic.
The AI agent must understand algorithms, data structures, complexity, and edge cases.
Typical prompt elements:
- Implement a shortest-path algorithm for a weighted graph. Explain the time complexity. Include tests for disconnected nodes and cycles.

SCEG-3 — Functional Semantics and Requirements Traceability
Focus: code implements the intended behaviour, not only a plausible behaviour.
The AI agent must connect requirements to implementation.
Typical prompt elements:
- Implement the booking logic according to the following business rules. For each rule, show where it is implemented and provide a test case.

SCEG-4 — Modular, Object-Oriented, and Component Design
Focus: internal software structure.
The AI agent must design modules, classes, objects, services, or components with good separation of concerns.
- This is the level where many AI-generated programs currently fail: they work locally but lack maintainable structure.

Typical prompt elements:
- Design a modular implementation with separate domain, service, repository, and interface layers. Use dependency

SCEG-5 — API, Data Model, and Semantic Interoperability
Focus: interfaces between components, services, systems, and organisations.
The AI agent must understand APIs not merely as endpoints, but as **contracts** supported by data models, schemas, semantics, versioning, and interoperability rules.
- This is one of the most important levels for complex and distributed systems and infrastructures.

Typical prompt elements:
- Design a REST API for experiment metadata registration. Provide OpenAPI specification, JSON schema, versioning strategy, error model, and semantic mapping to the common information model.

Typical prompt design elements -- SCEG Levels 6-9

SCEG-6 — Architecture Style and Layered System Composition
Focus: overall system organisation.
The AI agent must respect architectural styles, layers, deployment boundaries, and stakeholder concerns. ISO/IEC/IEEE 42010 is especially relevant here because it treats architecture descriptions through stakeholders, concerns, viewpoints, and views.

- This level is the software analogue of KCEG structural and ontological complexity.

Typical prompt elements:

- Propose a layered architecture for this system. Distinguish presentation, application, domain, data, integration, and infrastructure layers. Provide component diagram, dataflow diagram, and main architecture decisions.

SCEG-7 — Quality Attributes, Verification, and Risk Control
Focus: software quality beyond functionality.
The AI agent must generate or improve software with explicit quality targets: security, reliability, maintainability, performance, usability, safety, compatibility, flexibility, and testability. ISO/IEC 25010:2023 is a useful anchor because it defines a product quality model with nine quality characteristics and subcharacteristics for software and ICT products.

- This is where AI-generated software must move from “it runs” to “it is trustworthy.”

Typical prompt elements:

- Refactor this service to improve maintainability and security. Add unit tests, integration tests, input validation, logging, and explain how the result satisfies ISO 25010-style quality attributes.

SCEG-8 — Runtime, Deployment, Operations, and Observability
Focus: software as an operational system.
The AI agent must understand the target runtime environment: containers, cloud, CI/CD, monitoring, logging, scaling, rollback, configuration, secrets, and operational risks.

- This level is essential because programming AI agents increasingly generate not only code, but also deployment and infrastructure artefacts.

Typical prompt elements:

- Provide Dockerfile, docker-compose file, CI pipeline, health checks, structured logging, monitoring metrics, and deployment instructions for Kubernetes.

SCEG-9 — Evolutionary, Governance, and Socio-Technical Complexity
Focus: long-term software evolution in a real organisational environment.
This is the highest level. The AI agent is not only asked to write code, but to support sustainable engineering decisions.

- This is the software analogue of KCEG-9: not epistemic frontier knowledge, but **engineering-valid, evolvable, governed software knowledge embodied in executable systems**.

Typical prompt elements:

- Analyse this system for future evolution. Identify technical debt, API risks, maintainability risks, governance issues, compliance concerns, sustainability implications, and propose a staged improvement roadmap.

AI-Agent-ready Body of Knowledge clusters 1-6

BoK Cluster 1 — Problem Framing, Requirements and Promptable Specifications
SCEG link: SCEG-0 and SCEG-3.
This cluster teaches students to transform vague needs into structured specifications that can be used by humans and AI agents.
Prompt competence:
- Generate a solution for the following requirements.

BoK Cluster 2 — Programming, Algorithms and Local Code Correctness
SCEG link: SCEG-1 and SCEG-2.
This is the traditional programming foundation, but now it must include AI-generated code review.
Prompt competence:
- Implement the algorithm.
- Explain time and memory complexity.
- List edge cases.
- Generate unit tests.
- Identify possible failure modes.

BoK Cluster 3 — Software Design, Modularity and Object-Oriented / Component Thinking
SCEG link: SCEG-4.
This is where many programmers are weak and many AI-generated outputs fail.
Prompt competence:
- Refactor this code into modules with clear responsibilities.
- Separate domain logic, service logic, persistence and interface code.
- Explain coupling and cohesion decisions.
- Use dependency injection.

BoK Cluster 4 — API Design, Data Models and Semantic Interoperability
SCEG link: SCEG-5.
This should become a major curriculum element. It is often underdeveloped, but it is critical for modern software systems and AI-assisted development.
Prompt competence:
- Design an API contract, not only implementation code.
- Provide OpenAPI specification.
- Define request/response schemas.
- Define error responses.
- Explain field semantics.
- Map data fields to the common information model.
- Include versioning and backward compatibility strategy.

BoK Cluster 5 — Software and System Architecture
SCEG link: SCEG-6.
This should become the core curricula to ensure capability of generated code supervision and improvement.
Prompt competence:
- Propose three architecture options.
- For each option, describe:
  1. components;
  2. interfaces;
  3. data flow;
  4. deployment assumptions;
  5. quality-attribute trade-offs;
  6. risks;
  7. architecture decision record.

BoK Cluster 6 — Systems and Infrastructure Engineering and Lifecycle Thinking
SCEG link: SCEG-0, SCEG-6, SCEG-8, SCEG-9.
This is important when software is part of a larger technical, organisational, scientific, cyber-physical, cloud, or data infrastructure system.
Prompt competence:
- Analyse this software component as part of a larger system.
- Identify upstream and downstream dependencies.
- Define system and functional components boundary, interfaces, risks, verification needs and lifecycle concerns.
- Identify necessary cross-infrastructure services and interoperability aspects.

AI-Agent-ready Body of Knowledge clusters 7-12

BoK Cluster 7 — Software Quality, Verification, Validation and Testing
SCEG link: SCEG-7.
Prompt competence:
- Generate tests independently from the implementation.
- Create tests from requirements and edge cases.
- Identify missing tests.
- Assess the solution against quality attributes.

BoK Cluster 8 — Security, Privacy, Safety and Trustworthiness
SCEG link: SCEG-7 and SCEG-9.
Prompt competence:
- Perform a threat model for this design.
- Identify attack surfaces.
- Check input validation, authorization, dependency risk and secret handling.
- Avoid hardcoded secrets.
- Suggest secure alternatives.

BoK Cluster 9 — DevOps, Deployment, Cloud and Observability
SCEG link: SCEG-8.
Prompt competence:
- Generate deployment artefacts.
- Add CI/CD pipeline.
- Add health checks, logs and metrics.
- Separate configuration from code.
- Provide rollback and operational notes.

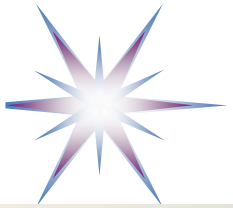
BoK Cluster 10 — Software Process, Team Engineering and Knowledge Management
SCEG link: SCEG-4 to SCEG-9.
Prompt competence:
- Produce code changes suitable for a pull request.
- Include design rationale, tests, documentation updates and review checklist.
- Identify what requires human review.

BoK Cluster 11 — Software Evolution, Maintenance, Governance and Sustainability
SCEG link: SCEG-9.
Prompt competence:
- Analyse this system for technical debt and evolution risks.
- Suggest a staged refactoring roadmap.
- Identify governance, compliance and sustainability concerns.

BoK Cluster 12 — AI-Agent-Aware Software Engineering Practice
SCEG link: all levels, but especially SCEG-3 to SCEG-9.
This is the new curriculum topic that should be added across the programme, adjusted to the course context.
Prompt competence:
- You are assisting with a software engineering task.
- Do not generate code first.
- First analyse requirements, assumptions, architecture, interfaces, risks and tests.
- Then propose an implementation plan.
- Only after approval generate code.

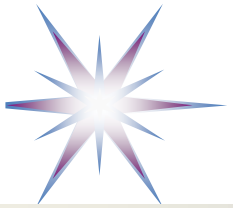
Example Course Elements DevOps and Agentic coding for Cloud based Applications

Example Project guidelines for Big Data applications and services



Q&A

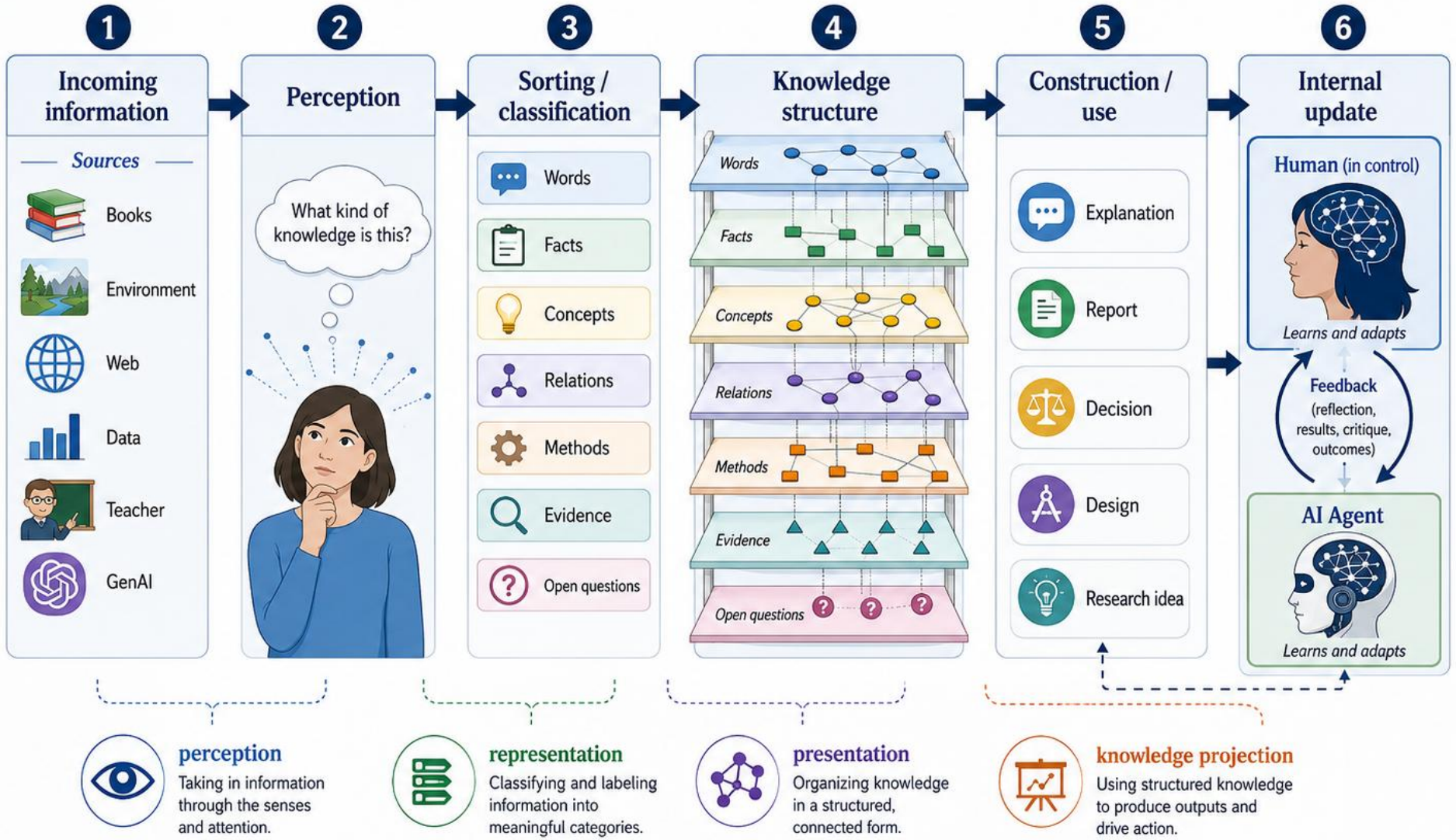
- Q: Can LLMs create knowledge? How LLM assistance can facilitate learning and education?
- A: Simple answer – LLM doesn't create knowledge the same way as a researcher potentially gains knowledge horizon when interacting with LLM Assistant
  - LLM as learning assistant can provide interactive support:
    - expose uncertainty
    - distinguish fact, inference, hypothesis, analogy, and speculation
    - help map concepts and claims
    - reveal gaps and contradictions
    - support verification
    - allow expert researchers to govern the process



Related information

Visualisation SCEG based Structured approach to Team based project development

KCEG as a Lens for Human–Agent Interaction

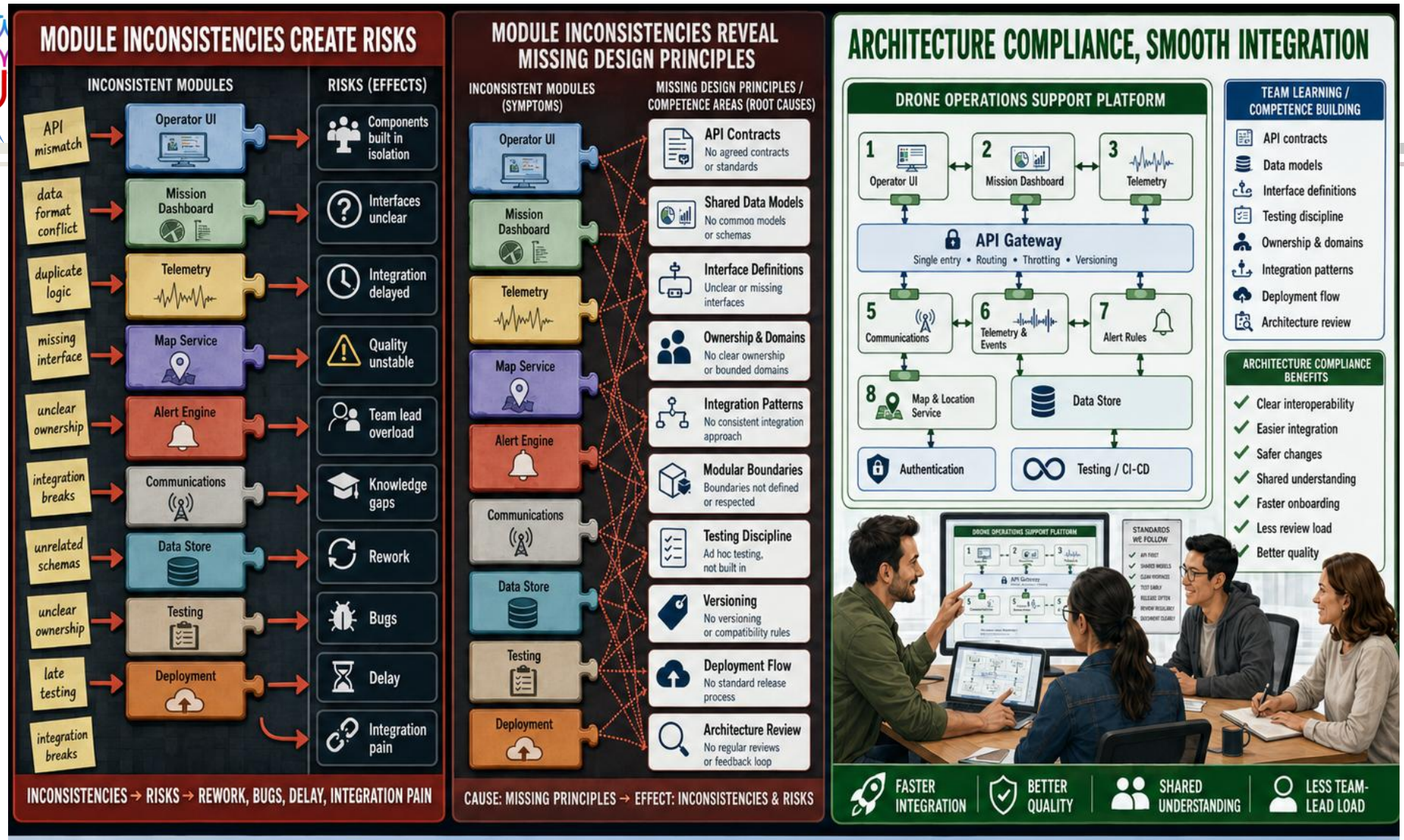


Using KCEG (Knowledge Complexity & Effective Guidance) Model

* For structured Perception, Learning and building Knowledge corpus

Software development company

- Architecture design principles and architecture based thinking are important for software components integration and interoperation



Software development company

- Architecture design principles and architecture based thinking are important for software components integration and interoperation
- Module inconsistency reveal missing applied architecture design principles

Conceptual chain for Human - AI interaction for HGLEP/KCEG

1. Knowledge source
- paper, dataset, prior dialogue, experiment, model, human expertise

2. Knowledge presentation
- text, diagram, table, formula, code, LLM response

3. Knowledge representation
- concepts, relations, ontology, claims, evidence, assumptions, methods

4. Knowledge perception / reconstruction
- human or AI agent interprets and reconstructs the meaning

5. Knowledge assessment
- KCEG evaluates level, completeness, uncertainty, reliability, novelty

6. Internalisation / agent memory update
- human updates personal epistemic model
- AI agent updates context, memory, graph, or retrieval state

7. Knowledge action
- explanation, critique, decision, experiment, paper writing, new prompt

8. Regeneration
- human or AI generates a new presentation

Knowledge is an actional element of the human cognitive composition
- Knowledge structure/direct actions
- Built-in/constructed/re-constructed knowledge structure defines learning efficiency and effective curricula

Basis for a generative loop:
presentation → perception → representation → assessment → memory → action → new presentation

Human - AI Epistemic Interaction Model

Human agent:

```
Personal epistemic model
   ├── semantic memory
   ├── methodological knowledge
   ├── tacit expertise
   ├── epistemic judgment
   └── goals / intentions
```

AI agent:

```
Agent knowledge state
   ├── context window
   ├── retrieved knowledge
   ├── latent model representation (ML internal)
   ├── symbolic/graph memory
   ├── tool outputs
   └── generated presentations
```

Interaction process:

```
Human intention
      ↓
Prompt / instruction presentation
      ↓
AI internal representation
      ↓
Generated knowledge presentation
      ↓
Human perception and reconstruction/representation
      ↓
KCEG assessment (by human/researcher)
      ↓
HGLEP-guided refinement
      ↓
Updated human and AI knowledge states
```

Methodological Framework Knowledge centered: Human-Governed LLM Epistemic Projection for Frontier Research

- Human-Governed LLM Epistemic Projection for Frontier Research (HGLEP – HGLEP-FR)
- **LLM-assisted research** is most valuable when the LLM acts as an **epistemic projection instrument**: it transforms uneven, large-scale, and partially reliable accessible knowledge into structured **research candidates**, while the expert researcher governs interpretation, verification, and integration into validated knowledge.
- Knowledge-Complexity and Epistemic-Governance Model of LLM-Assisted Research (KCEG)
- LLM should not be treated as autonomous producers of validated research knowledge.
  - In frontier research, LLM can **project, reorganize, and structure LLM-accessible knowledge** around a researcher's question.
  - The resulting outputs are not final truths, but **candidate knowledge structures** that require expert interpretation, methodological verification, and epistemic validation.
- The researcher remains the **epistemic governor** of the process.

Data, Information, Knowledge – Knowledge types

- Level 1. Physical Reality
  - Starting point of all data, information, knowledge: objects, events, processes, people
- Level 2. Data
  - Data are symbolic observations about reality.
- Level 3. Information
  - Information is interpreted data placed into context.
- Level 4. Structured Information
  - Information becomes structured by relations: ERP, CRM, ontology, workflow, knowledge graph
- Level 5. Operational Knowledge
  - First level that can be qualified as knowledge: Provides a basis for what should be done: rules. protocols
- Level 6. Conceptual Knowledge
  - Basic/foundational concepts why things exist: architecture, scientific theory, domain models
- Level 7. Methodological Knowledge
  - Methodology to create knowledge: scientific & engineering methods
- Level 8. Epistemic Knowledge
  - Basis for knowledge quality and validation, scientific reasoning and peer review
- Level 9. Frontier Knowledge
  - Generative Knowledge

The current Knowledge hierarchy can be applied equally to:

- business information systems,
- enterprise knowledge management,
- AI agents,
- scientific research,
- LLMs,
- human cognition.

Knowledge Complexity Aspects

- General, linked to knowledge presentation
  - Vocabulary and semantics describe **how knowledge is expressed** (in words and terms).
  - Ontology and structure describe **how it is organised**.
  - Inference and methodology describe **how it works and how it is produced**.
  - Epistemic measures describe **how reliable, novel, uncertain, or contested it is**.
- For humans, complexity is strongly tied to prior knowledge, cognitive load, and research judgment.
- For LLMs, complexity is tied to representation, grounding, context, retrieval, reasoning depth, and verifiability.
- The measure cannot be a single score but a **complexity vector** across these dimensions.

KCEG Perception by Expert and Novice

Mechanism: Perception is holistic, understanding is selective reconstruction

- **1. Real object/event/system**
  The full THING EXISTS with many properties: visible, functional, structural, causal, methodological, historical, epistemic.
- **2. Sensory/perceptual intake**
  The person receives SIGNALS: visual form, words, diagrams, behaviour, motion, numbers, names, artefacts.
  - ----------
- **3. Cognitive reconstruction**
  The MIND MAPS signals into existing *schemas*: vocabulary, concepts, relations, structures, methods, causal models, epistemic status.
- **4. Expressed understanding**
  The person explains, draws, predicts, classifies, evaluates, or acts.
- **Hypothesis: Gap between layers 2 and 3/4**.
  - The person may "see" the whole event, but their **model of what is significant** is constrained by their available cognitive schemas.
  - This is close to the expert-novice literature review: experts detect meaningful patterns that novices overlook, and experts organise knowledge around deep principles rather than surface attributes.